

Network Attack Detection Analysis

Vítor Hugo Magnus Oliveira - 00341650
Thomas Schneider Wiederkehr - 00342606
Leonardo Gonzatti Heisler - 00344125

Ciência de Dados
Semestre 2026/1

Sumário

1	Objetivo	3
2	Dataset	3
2.1	Origem e composição	3
2.2	Limpeza e pré-processamento	4
2.3	Geração de timestamps sintéticos	4
3	Metodologia	5
3.1	Pipeline geral	5
3.2	Configuração dos modelos	5
3.3	Dashboard interativo	6
4	Resultados	7
4.1	Comparação de desempenho	7
4.2	Análise por classe: Random Forest	7
4.3	Importância de features (Random Forest)	8
5	Conclusões	8
5.1	Interpretação crítica dos resultados	8
5.2	Impacto para um SOC real	9
5.3	Limitações da abordagem de tempo sintético	9

Resumo

Este relatório descreve o projeto **Network Attack Detection Analysis**, uma plataforma de análise para Centros de Operações de Segurança (SOC) desenvolvida sobre o dataset CICIDS-2017. O pipeline abrange carregamento e limpeza dos dados, geração de timestamps sintéticos para simulação operacional, treinamento comparativo de três classificadores (Decision Tree, Random Forest e XGBoost) e um dashboard interativo em Streamlit que simula o monitoramento de tráfego em tempo real. O melhor modelo obtido foi o **Random Forest**, com **F1 macro de 0,9981** e acurácia de **99,94%** no conjunto de teste, demonstrando a viabilidade de um sistema de detecção automatizada de intrusões baseado em aprendizado de máquina para os cenários de tráfego benigno, DDoS e PortScan. É possível visualizar a Dashboard e interagir com um passo a passo do pipeline por um notebook do *Google Colab* que pode ser acessado no seguinte link: <https://colab.research.google.com/drive/173iGI5XBFS84VeBb2dzX2ITvanQlp6mW?usp=sharing>

1 Objetivo

O projeto tem como objetivo a construção de um *pipeline de ponta a ponta* para detecção de ataques de rede, simulando as condições operacionais de um Centro de Operações de Segurança (SOC). Os objetivos específicos são:

- Implementar um pipeline reprodutível de ingestão, limpeza e engenharia de atributos sobre o dataset CICIDS-2017.
- Treinar e comparar três modelos de classificação supervisionada para distinguir tráfego benigno de dois cenários de ataque: DDoS e PortScan.
- Gerar timestamps sintéticos plausíveis que permitam simular análises temporais típicas de SOC (timelines, picos de incidente, sazonalidade diurna) sem distorcer as proporções reais das classes.
- Disponibilizar um dashboard interativo (Streamlit + Plotly) com 6 páginas funcionais, incluindo um painel operacional de SOC com janela temporal deslizante e índice de risco em tempo real.

2 Dataset

2.1 Origem e composição

O projeto utiliza o dataset **CICIDS-2017** (*Canadian Institute for Cybersecurity Intrusion Detection System 2017*), amplamente empregado em pesquisas de detecção de intrusão. Foram utilizados três arquivos CSV da coleção *MachineLearningCVE*:

- `Monday-WorkingHours.pcap_ISCX.csv`: tráfego exclusivamente benigno.
- `Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv`: ataques DDoS e tráfego benigno.
- `Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv`: ataques de PortScan e tráfego benigno.

Após a concatenação e normalização dos rótulos (que apresentam variações de grafia entre arquivos), foram mantidas apenas as três classes-alvo: **BENIGN**, **DDoS** e **PortScan**. O dataset final contém **71 atributos** de fluxo de rede (mais o rótulo), extraídos pela ferramenta CICFlowMeter.

2.2 Limpeza e pré-processamento

O módulo `src/preprocessing.py` aplica as seguintes etapas, na ordem indicada:

1. **Coerção numérica:** todas as colunas de feature são convertidas para numérico; valores não conversíveis tornam-se NaN.
2. **Tratamento de infinitos:** valores `Inf/-Inf` (comuns em *Flow Bytes/s*) são substituídos por NaN.
3. **Imputação:** NaN restantes preenchidos com a mediana da respectiva coluna (robusto a outliers).
4. **Remoção de duplicatas** exatas.
5. **Remoção de colunas constantes** (variância zero).
6. **Codificação de rótulos** via `LabelEncoder` com ordem canônica [**BENIGN**, **DDoS**, **PortScan**].

2.3 Geração de timestamps sintéticos

Aviso importante: o dataset CICIDS-2017 não possui uma linha temporal contínua adequada para monitoramento operacional. A coluna **Timestamp** adicionada pelo módulo `src/feature_engineering.py` é **100% sintética** e existe exclusivamente para simulação de análises de SOC (timelines, picos, sazonalidade). Ela não representa o tempo real dos fluxos capturados.

A estratégia de geração de timestamps é parametrizada em `config.SYNTHETIC_TIME` e segue a lógica a seguir:

- **Janela simulada:** 5 dias consecutivos a partir de 2017-07-03 (segunda-feira), com semente aleatória 42 para reprodutibilidade.
- **Tráfego benigno (BENIGN):** distribuído por todos os 5 dias segundo um *padrão diurno gaussiano*. A função `_diurnal_weights` gera pesos de probabilidade por hora do dia (0–23) como uma curva em sino centrada no meio do expediente comercial (`business_hours = (8, 18)`), com piso mínimo de 0,05 para evitar janelas completamente vazias à noite.
- **Classes de ataque (DDoS / PortScan):** os eventos são divididos em duas frações:
 - **85% concentrados** em janelas de incidente pré-definidas, criando picos de atividade plausíveis na timeline.
 - **15% espalhados** como ruído de fundo ao longo de todos os dias (maior realismo).

As janelas de incidente configuradas são:

- DDoS: dia 1 das 10h às 12h; dia 3 das 14h às 16h.
- PortScan: dia 0 das 9h às 11h; dia 2 das 13h às 15h; dia 4 das 16h às 18h.
- **Colunas derivadas** adicionadas ao DataFrame: `Timestamp`, `date`, `hour`, `weekday`, `day_label`.

A feature `hour` derivada dos timestamps sintéticos é incluída como atributo de entrada nos modelos (sendo a 71ª feature). Isso implica que qualquer importância atribuída a `hour` reflete a distribuição sintética e não uma característica real dos fluxos (limitação discutida na Seção 5).

3 Metodologia

3.1 Pipeline geral

O pipeline, orquestrado por `src/pipeline.py`, executa as seguintes etapas em sequência:

1. Carregamento dos CSVs reais via `src/data_loader.py` (fallback para dados sintéticos caso os CSVs não estejam presentes).
2. Amostragem estratificada: caso o dataset ultrapasse `MAX_SAMPLES = 120.000` registros, aplica-se uma subamostra proporcional por classe.
3. Geração de timestamps sintéticos (`src/feature_engineering.py`).
4. Pré-processamento e codificação (`src/preprocessing.py`).
5. Divisão treino/teste estratificada: proporção 75/25, `random_state = 42`.
6. Treinamento dos três modelos.
7. Avaliação e seleção automática do melhor modelo pela métrica **F1 macro** (`config.SELECTION_METRIC`).
8. Serialização do bundle (`models/models_bundle.joblib`) e exportação das métricas (`models/metrics.json`).

3.2 Configuração dos modelos

Os três classificadores são instanciados com os hiperparâmetros centralizados em `config.MODEL_PARAMS`:

Hiperparâmetro	Valor	Observação
Decision Tree		
max_depth	18	Limita overfitting
class_weight	balanced	Compensa desbalanceamento
Random Forest		
n_estimators	200	Número de árvores
max_depth	22	Árvores mais profundas que DT
class_weight	balanced	Compensa desbalanceamento
n_jobs	-1	Paralelismo total
XGBoost		
n_estimators	300	Boosting iterations
max_depth	8	Árvores mais rasas (regularização)
learning_rate	0,2	Passo de atualização
subsample	0,9	Subamostragem por iteração
colsample_bytree	0,9	Subamostragem de features
tree_method	hist	Algoritmo aproximado (eficiência)
objective	multi:softprob	Classificação multiclasse

3.3 Dashboard interativo

A aplicação `app/dashboard.py` (Streamlit + Plotly) expõe 6 páginas navegáveis pela barra lateral:

1. **Overview:** KPIs executivos (total de registros, % de ataques, tráfego benigno, DDoS e PortScan detectados, melhor modelo); gráfico de pizza (donut) de proporção por classe e área temporal de volume de eventos.
2. **Exploração dos Dados (EDA):** distribuição de classes em barras; histograma e boxplot interativos por feature (escala log opcional); matriz de correlação de Pearson.
3. **Monitoramento Temporal:** timeline de eventos por classe com granularidade ajustável (30 min/1 h/3 h/6 h); volume por hora do dia; ataques por hora; eventos por dia; comparativo DDoS vs. PortScan; tabela de períodos de pico.
4. **Feature Importance:** gráfico de barras horizontais com as importâncias de atributos do modelo selecionado; tabela completa do ranking.
5. **Simulador de Predição:** sliders interativos para as 8 features mais importantes; presets por perfil de classe (mediana global, BENIGN, DDoS, PortScan); exibição da classificação prevista e probabilidades por classe.
6. **Security Operations Center:** painel com janela temporal deslizante (1 h, 2 h, 3 h ou 6 h), navegação por botões e *auto-play*; KPIs da janela (eventos, DDoS, PortScan, índice de risco 0–100 com nível NORMAL/ELEVADO/CRÍTICO); gauge de risco; tabela de alertas ativos; feed dos eventos recentes.

A sidebar disponibiliza filtros globais (classes, período e faixa de horas) e botão de re-treinamento sob demanda.

4 Resultados

4.1 Comparação de desempenho

Os modelos foram avaliados em um conjunto de teste estratificado composto por **30.000 fluxos**: 26.638 BENIGN, 1.500 DDoS e 1.862 PortScan. A Tabela 1 apresenta as métricas agregadas.

Tabela 1: Comparação de métricas entre os classificadores (conjunto de teste).

Modelo	Accuracy	Precision	Recall	F1 macro	F1 weighted
Decision Tree	0,9991	0,9970	0,9978	0,9974	0,9991
Random Forest	0,9994	0,9970	0,9991	0,9981	0,9994
XGBoost	0,9993	0,9974	0,9987	0,9980	0,9993

Precision, Recall e F1 macro = média entre as 3 classes. Seleção automática pela métrica F1 macro (config.SELECTION_METRIC).

O **Random Forest** foi selecionado automaticamente como o melhor modelo. A diferença entre RF (F1 macro 0,9981) e XGBoost (0,9980) é marginal, inferior a 0,01% em F1 macro, indicando que ambas as abordagens de *ensemble* atingem desempenho virtualmente equivalente neste dataset. A Decision Tree, apesar de ser um classificador mais simples, também alcança métricas expressivas (F1 macro 0,9974), evidenciando a separabilidade intrínseca das classes no espaço de features de fluxo.

4.2 Análise por classe: Random Forest

A Tabela 2 detalha as métricas individuais do Random Forest para cada classe do conjunto de teste.

Tabela 2: Métricas por classe (Random Forest).

Classe	Precision	Recall	F1	Suporte
BENIGN	0,9999	0,9994	0,9996	26.638
DDoS	0,9987	0,9980	0,9983	1.500
PortScan	0,9925	1,0000	0,9963	1.862

O destaque é o **Recall = 1,0000 para PortScan**: todos os 1.862 casos de varredura de portas foram corretamente identificados, sem nenhum falso negativo, propriedade crítica para um SOC, pois um PortScan não detectado pode preceder uma intrusão. A matriz de confusão correspondente é:

$$\mathbf{C}_{\text{RF}} = \begin{pmatrix} 26.622 & 2 & 14 \\ 3 & 1.497 & 0 \\ 0 & 0 & 1.862 \end{pmatrix} \begin{array}{l} \leftarrow \text{BENIGN (pred.)} \\ \leftarrow \text{DDoS (pred.)} \\ \leftarrow \text{PortScan (pred.)} \end{array}$$

Os únicos erros observados são 14 fluxos BENIGN classificados como PortScan (falsos positivos), 2 BENIGN como DDoS e 3 DDoS como BENIGN. São volumes diminutos num universo de 30.000 amostras.

4.3 Importância de features (Random Forest)

A Tabela 3 lista as 10 features mais relevantes extraídas do modelo Random Forest via `feature_importances_`.

Tabela 3: Top 10 features por importância (Random Forest, de 71 no total).

Rank	Feature	Importância
1	Fwd Packet Length Max	0,0855
2	Avg Fwd Segment Size	0,0814
3	Fwd Packet Length Mean	0,0776
4	Subflow Fwd Bytes	0,0659
5	Total Length of Fwd Packets	0,0579
6	Packet Length Mean	0,0497
7	Total Fwd Packets	0,0361
8	Average Packet Size	0,0337
9	Subflow Fwd Packets	0,0298
10	Flow Duration	0,0281

As features dominantes são relacionadas ao **tamanho dos pacotes no sentido direto (forward)** da conexão: *Fwd Packet Length Max*, *Avg Fwd Segment Size* e *Fwd Packet Length Mean* lideram o ranking com importâncias acima de 0,075. Isso é consistente com o comportamento esperado dos ataques:

- **DDoS**: gera rajadas de pacotes com payloads volumosos (*fwd length* alto) e alta cadência.
- **PortScan**: envia pacotes mínimos (SYN isolados), distinguindo-se pelo tamanho muito reduzido e pela ausência de tráfego reverso (*bwd* = 0).
- **BENIGN**: apresenta fluxos bidirecionais equilibrados com payloads de tamanho médio.

5 Conclusões

5.1 Interpretação crítica dos resultados

Os três modelos avaliados atingiram métricas excepcionais ($F1 \text{ macro} > 0,997$), confirmando que os atributos de fluxo do CICIDS-2017 são altamente discriminativos para as classes BENIGN, DDoS e PortScan. O **Random Forest** se destacou marginalmente, oferecendo o melhor equilíbrio entre Precision, Recall e $F1 \text{ macro}$, com a vantagem adicional de Recall perfeito para PortScan, propriedade de alto valor operacional.

A proximidade entre Random Forest e XGBoost (diferença $< 0,01\%$ em $F1 \text{ macro}$) é esperada: ambos são modelos de ensemble baseados em árvores, e o ganho do boosting sobre o bagging é frequentemente superado pela regularização implícita do bagging em datasets limpos e bem separados.

Porém, estes resultados também relevam que estamos trabalhando com um dataset extremamente simples e não podemos esperar tamanha acurácia em um cenário real de segurança.

5.2 Impacto para um SOC real

Um sistema com F1 macro próximo a 1,000 e Recall = 1,0 para PortScan seria, em princípio, extremamente valioso em produção: detecta virtualmente todos os ataques com uma taxa de falsos positivos muito baixa. No entanto, sua adoção em um SOC real exige considerar os pontos a seguir.

Pontos favoráveis:

- Inferência em tempo real viável (os modelos de árvore são rápidos e serializáveis).
- Interpretabilidade via Feature Importance facilita a explicação das decisões para analistas de segurança.
- O pipeline é reproduzível e modular, permitindo re-treinamento sob demanda com novos dados.

Pontos de atenção:

- **Generalização para tráfego fora da distribuição:** o CICIDS-2017 foi capturado em ambiente de laboratório controlado em 2017. Ataques modernos podem ter perfis de fluxo distintos dos representados no dataset.
- **Dois vetores de ataque apenas:** o modelo atual discrimina somente DDoS e PortScan. Um SOC real enfrenta dezenas de categorias (exploits, C&C, exfiltração, ransomware, etc.).
- **Deriva de distribuição (*concept drift*):** o tráfego de rede evolui ao longo do tempo; o modelo exigiria re-treinamento periódico com dados rotulados recentes.

5.3 Limitações da abordagem de tempo sintético

A geração de timestamps sintéticos é a limitação mais significativa do projeto para uso operacional:

1. **Desconexão temporal:** os timestamps não correspondem ao instante real de captura dos fluxos. Análises de janela temporal (frequência de ataques, sazonalidade) refletem a distribuição escolhida pelo projetista, não o comportamento real dos atacantes.
2. **Feature hour contaminada:** a 71ª feature (**hour**) é derivada do timestamp sintético. Se o modelo aprendeu a associar certas horas a certos ataques, essa associação é artificial e pode produzir erros caso aplicado a dados com timestamps reais em horários distintos dos configurados.
3. **Otimismo das métricas:** embora as métricas elevadas sejam em grande parte justificadas pela separabilidade real das classes no espaço de features, a ausência de variação temporal autêntica pode inflar ligeiramente os resultados ao eliminar correlações temporais espúrias que ocorrem em tráfego real.

Para um cenário de produção, recomenda-se:

- Substituir os timestamps sintéticos pelas marcas de tempo originais dos pcaps do CICIDS-2017 (ou de capturas mais recentes).
- Excluir a feature **hour** do conjunto de treino caso os timestamps não sejam fidedignos.

- Ampliar o escopo para os demais tipos de ataque do CICIDS-2017 (Brute Force, Web Attacks, Infiltration, Botnet) e adotar um regime de validação cruzada temporal (*walk-forward*) ao invés do split aleatório.

Referências

- Sharafaldin, I., Habibi Lashkari, A., Ghorbani, A. A. *Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization*. In: ICISSP 2018. <https://www.unb.ca/cic/datasets/ids-2017.html>
- Pedregosa, F. et al. *Scikit-learn: Machine Learning in Python*. JMLR 12, pp. 2825–2830, 2011.
- Chen, T., Guestrin, C. *XGBoost: A Scalable Tree Boosting System*. KDD 2016.